

APACHE DORIS 3.0.8

复习题答案

第四章：查询执行与优化

20 道综合题参考答案。每题包含一句话结论、底层因果链、得分关键词与常见误区。

配套资料：notes-04

基准版本：Apache Doris 3.0.8

整理日期：2026-08

答案导航与评分

建议先合上答案册口述：一句话结论、对象关系、数据流、一个可观测证据。每题 10 分：结论 2 分，核心机制 4 分，因果链 2 分，边界或诊断方法 2 分。

题号	核心主题	关键词
1	声明式SQL	what / how、物理路线
2	计划层次	Logical、Physical、Fragment、Instance
3	RBO与CBO	规则、统计、代价
4	统计与Broadcast	估算、复制放大、Hash内存
5	NDV与Cardinality	选择率、误差传播
6	Fragment与Exchange	Sink、ExchangeNode、分布
7	三层执行	MPP、Pipeline、Vectorization
8	任务调度	Blocked、Yield、Backpressure
9	多层裁剪	Partition、Tablet、Column、ZoneMap
10	扫描反模式	函数、SELECT *
11	Hash Join	Build、Probe、Dependency
12	Join分布	Broadcast、Shuffle、Bucket、Colocate
13	倾斜与膨胀	max/avg/min、many-to-many
14	两阶段聚合	Local、Exchange、Global
15	TopN	M log N、OFFSET
16	Runtime Filter	IN、Bloom、MinMax
17	并行度	延迟、吞吐、状态复制
18	内存超限	Hash、Agg、Sort、根因
19	诊断工具	EXPLAIN、Profile
20	第一处异常	沿数据流、单变量验证

1. 为什么说 SQL 只描述‘要什么’，而物理执行计划决定‘怎么做’？

SQL是声明式接口，只规定结果语义；扫描范围、Join顺序、数据分布、聚合阶段和并行方式由优化器与执行引擎决定。

因果链

- 同一SQL语义可能有多个等价关系计划。
- 每个逻辑Join又可能对应Broadcast、Shuffle等不同物理实现。
- 不同路线读取、计算和传输的数据量不同，所以性能可能相差数量级。
- 声明式设计让存储布局、统计和集群拓扑变化时，业务SQL无需描述节点级执行。

得分关键词

声明式；语义等价；逻辑计划；物理计划；少读少传；业务与执行解耦

常见误区：把SQL文本顺序当成固定执行顺序。Inner Join等结构可能被Nereids重排。

2. 逻辑计划、物理计划、PlanFragment 和 Fragment Instance 有什么区别？

逻辑计划描述关系运算，物理计划确定算法和分布，Fragment是可下发的计划段，Instance是Fragment在具体BE上的运行副本。

从抽象关系到真实运行



一个Fragment可在多个BE上产生多个Instance，从而获得横向并行；每个Instance内部还会拆成Pipeline和PipelineTask。

得分关键词

关系代数；算法；数据分布；Fragment模板；Instance副本；PipelineTask

常见误区：把一个Fragment等同于一台固定BE。Fragment是模板，运行时可以有多个Instance。

3. RBO 与 CBO 各自解决什么问题，为什么不能互相替代？

RBO用确定性规则消除明显浪费；CBO借助统计信息在多个合理方案中比较成本。前者不擅长规模选择，后者也需要先经过语义安全的计划清理。

维度	RBO	CBO
典型工作	谓词下推、列/分区裁剪、常量折叠	Join顺序、Build Side、数据分布
依据	规则是否匹配	统计与代价模型
风险	规则覆盖有限	统计过期、倾斜和相关性
得分关键词	规则改写；确定性；统计信息；候选计划；成本；先RBO后CBO	

常见误区：认为CBO可以自动修复所有坏SQL。无法推导的分区条件、错误Join语义仍需从SQL或模型解决。

4. 为什么统计信息过期可能把合理的 Broadcast Join 变成内存事故？

Broadcast成本按过滤后的Build Side大小乘以Join Instance数量放大；如果CBO低估小侧，网络与每实例Hash表都会按真实大数据量复制。

```
: Build=200MB, Instance=10 -> 2GB
: Build=5GB, Instance=10 -> 50GB
Join InstanceHash
```

应对方法是比较EXPLAIN估算与Profile的Build InputRows、HashTable Memory、RemoteBytes，更新统计后重新让CBO选择；不是立刻固定另一个Hint。

得分关键词	过滤后大小；N×R；Hash表复制；估算与实际；ANALYZE；重新规划
-------	--------------------------------------

常见误区：只看维表原始名称就认为它一定小。应看过滤和投影后的真实行数与行宽。

5. 列 NDV 与算子 Cardinality 有什么区别？误差为什么逐层放大？

NDV是列中不同值数量，Cardinality是计划节点预计或实际输出行数；上游Cardinality会成为下游Join、Aggregate和Sort的输入，因此误差会连续传播。

```
status NDV = 4 # 4
Filter cardinality = 10,000,000 # 1
```

- NDV=4不代表四个值各占25%，热点值会破坏均匀假设。
- 日期与状态可能相关，多个条件选择率不能总是简单相乘。
- Filter低估会导致Join分布、聚合Hash表和Sort内存继续低估。

得分关键词 NDV；节点行数；选择率；均匀分布；列相关性；误差传播

常见误区：把cardinality字段理解为实际行数。EXPLAIN中通常是估算，Profile才是实际。

6. PlanFragment 为什么常在 Exchange 附近切分？

Exchange改变数据所在节点或分布属性，上游由DataStreamSink发送，下游由ExchangeNode接收，因此自然形成两个可独立下发的Fragment。

方式	改变	用途
Gather	多路汇聚到少数接收端	最终结果、全局TopN
Hash	相同Key到同一接收端	Join、Global Aggregate
Broadcast	完整复制一侧	大表本地Probe小表Hash

得分关键词 分布边界；DataStreamSink；ExchangeNode；Fragment；Gather；Hash；Broadcast

常见误区：认为Exchange一定是坏事。全局计算需要必要交换，重点是交换前是否缩小以及交换量是否合理。

7. MPP、Pipeline 与向量化分别解决哪个层次的问题？

MPP扩展多机算力，Pipeline在单BE内调度有限线程，向量化让单个算子按列式Block批量利用CPU。

机制	范围	核心收益
MPP	BE之间	横向扩展、数据本地并行
Pipeline	BE内部	任务与线程解耦、阻塞时让出Worker
向量化	算子内部	减少调用、提高Cache命中、使用SIMD

得分关键词 多机；线程池；Task；Block；列式；SIMD

常见误区：把Pipeline与向量化当作同一个功能。一个负责调度，一个负责每批数据的计算方式。

8. PipelineTask 阻塞时为什么应该让出 Worker？背压解决什么问题？

Task等待依赖、网络或I/O时不需要CPU，让出Worker可执行其他Ready任务；背压在下游消费不过来时限制上游生产，防止缓冲无限增长。

任务阻塞不等于线程必须阻塞



得分关键词

Task/Worker解耦；Dependency；Yield；有界缓冲；背压；内存控制

常见误区：认为Pipeline消灭了阻塞。它只是把阻塞表示为依赖，并尽量不让阻塞Task占住线程。

9. 分区、Tablet、列裁剪和 ZoneMap 的作用层级有什么不同？

它们从FE粗粒度元数据到BE细粒度数据页逐层跳过：先裁Partition，再定位Bucket/Tablet，再不读无用列，最后用Min/Max排除Segment或Page。

裁剪越早，下游需要处理的对象越少



EXPLAIN可检查partitions=N/M、tablets=N/M和PREDICATES；ZoneMap等BE细粒度收益更多在Profile过滤指标中体现。

得分关键词

FE粗裁剪；BE细裁剪；Partition；Bucket；Column；Min/Max；EXPLAIN/Profile

常见误区：把分区和分桶当成同一层。Partition是逻辑范围，Tablet是分区内部的物理分片。

10. 为什么列上包裹函数、使用 SELECT * 会削弱扫描效率？

函数可能让优化器无法把条件反推成原始列范围；SELECT *则要求所有输出列都保留，使列式存储无法跳过宽列。

```
--
order_date >= '2026-08-01' AND order_date < '2026-09-01'
```

```
--
DATE_FORMAT(order_date, '%Y-%m') = '2026-08'
```

- 原始列条件可用于Partition Range、ZoneMap与统计估算。
- SELECT *会增加磁盘读取、解压、Block宽度和Exchange字节。

得分关键词

可推导谓词；原始列；半开范围；列裁剪；宽列；I/O字节

常见误区：只比较返回行数。宽表即使返回行少，读取全部大字符串列仍可能很贵。

11. Hash Join 的 Build 与 Probe 如何协作？为什么小侧做 Build？

Build Side先构建Join Key到行位置的Hash表，Probe Side随后流式查找；较小Build能降低Hash构建、内存峰值和依赖等待。

- Build Pipeline完成后设置Dependency Ready。
- Probe Task按Block批量计算Hash并验证等值条件。
- Build大小应按过滤与投影后的真实数据判断。

- Hash表还包含桶、指针、Key和Arena，内存通常大于原始列大小。

得分关键词

Build Hash表；Probe流式；Dependency；过滤后行数；行宽；内存结构

常见误区：认为SQL右表永远是业务意义上的小表。CBO可重排，物理计划中的Build侧才是分析对象。

12. 四种 Join 分布策略的成本与约束是什么？

Broadcast复制右侧，Shuffle移动两侧，Bucket Shuffle复用一侧分桶，Colocate让对应Bucket预先同址；网络依次可近似为 $N \times R$ 、 $L+R$ 、 R 、 0 。

策略	成本	主要条件
Broadcast	$N \times R$	Build过滤后足够小
Shuffle	$L+R$	两侧按Join Key重新Hash
Bucket Shuffle	R	一侧Bucket分布可复用
Colocate	0	同组、同规则、同址且稳定

得分关键词

 $N \times R$ ； $L+R$ ；复用Bucket；同址；数据布局；网络

常见误区：认为Colocate无条件最快。Bucket太少会限制并行度，Group不稳定也可能退化。

13. Join Key 倾斜与 Join 结果膨胀有什么区别？

倾斜是工作分配不均，膨胀是Join语义产生了过多结果；前者看max/avg/min差异，后者看RowsProduced相对输入异常增大。

```
: ProbeRows avg=10M, max=80M, min=1M
: Key=11000Key=11000 -> 100
```

- 倾斜使单个Straggler拖住阶段，即使总行数不异常。
- 膨胀常来自多对多、维表Key不唯一或Join条件缺失。
- 二者可同时发生：热点Key既集中又产生笛卡尔式匹配。

得分关键词

工作不均；结果基数；max/avg/min；RowsProduced；多对多；唯一性

常见误区：看到某Task很慢就只提高并行度。相同Hash Key仍会去同一个目的地。

14. 为什么局部聚合应尽量发生在 Exchange 之前？

Local Aggregate先把明细压成可合并状态，Exchange只传每个Group的局部状态，再由Global Aggregate合并，从而减少网络和下游Hash工作。

低基数Group的缩减效果最明显



高基数Group Key可能让1亿行仍输出接近1亿组，局部Hash表更大且缩减率很低；因此要同时检查NDV、InputRows、RowsProduced与HashTable Memory。

得分关键词

Local/Global；可合并状态；Exchange前缩减；NDV；Hash表；Input/Output

常见误区：把执行层Local Aggregate与数据模型的PREAGGREGATION混为同一概念。

15. TopN 为什么比全量 Sort 便宜？大 OFFSET 为什么仍慢？

TopN只维护N个最优候选，近似 $O(M \log N)$ 和 $O(N)$ 内存；大OFFSET必须保留OFFSET+LIMIT个候选并丢弃前面结果，成本随OFFSET增长。

- 分布式执行可先做Local TopN，再汇聚候选做Global TopN。
- 排序Key匹配表Key前缀时可利用已有顺序。
- 两阶段读取先读排序列，再回读少量完整行。
- 连续翻页可用稳定排序Key的游标条件替代巨大OFFSET。

得分关键词

$M \log N$ ；候选堆；Local/Global；OFFSET+LIMIT；Key前缀；延迟物化

常见误区：把ORDER BY LIMIT的执行TopN与TOPN()近似频次聚合函数混为一谈。

16. IN、Bloom、Min/Max Runtime Filter 各有什么性质？

IN是小集合精确匹配；Bloom能确定不存在但允许假阳性；Min/Max用值域包络排除范围外数据，成本低但对离散宽范围效果弱。

类型	正确性与代价	边界
IN	精确，无误判	集合过大时构建、传输与查找变贵
Bloom	无假阴性，可有假阳性	通过者仍需Join验证
Min/Max	结构小、易结合ZoneMap	min/max跨度很大时选择性差

Runtime Filter来自执行期Build实际数据，应用到Probe Scan；Outer Join等语义可能限制下推。Global Filter还需要合并各Instance的Partial Filter。

得分关键词	动态生成；Build→Probe；精确集合；假阳性；无假阴性；值域；Global Merge
-------	--

常见误区：认为Bloom命中就代表一定能Join成功。Bloom只说明可能存在。

17. 为什么提高 parallel_pipeline_task_num 可能降低吞吐并增加内存？

更多Task能降低CPU密集型单查询延迟，但也增加状态分片、Local Exchange、队列和调度，并让单条查询占用更多核心，从而挤压并发查询。

- CPU未满足且Task行数均匀时，提高并行度可能有效。
- I/O或网络已满、依赖等待高、数据倾斜时，提高并行度通常无效。
- 点查询和高并发场景更关心每查询资源成本，而不是单查询最大并行。
- 优先用SQL级SET_VAR做单变量实验，避免直接全局修改。

得分关键词

单查询延迟；系统吞吐；Task状态复制；Local Exchange；CPU利用；单SQL实验

常见误区：以CPU核心数为理由把所有查询并行度都设到最大。总活跃Task还会乘以查询并发、Fragment与BE数量。

18. 遇到 MEM_LIMIT_EXCEEDED 为什么不应先无限提高内存？

内存超限通常是计划或数据规模问题的结果；提高限制可能把失败推迟，或让单查询占满BE并伤害其他负载。

优先排查

- Hash Join Build是否被低估或错误Broadcast。
- Group Key NDV是否异常，聚合Hash表是否接近输入规模。
- Sort是否缺少LIMIT，OFFSET是否过大。
- Join是否多对多膨胀，SELECT *是否扩大行宽。
- 并行度和并发查询是否复制过多状态。

确认查询本来就需要更大内存后，可做受控调整；Spill用磁盘换取继续执行能力，延迟通常更高。

得分关键词

Memory Tracker；Hash/Agg/Sort；Broadcast；行宽；并行度；并发；Spill

常见误区：把Spill当成加速功能。它主要是稳定性和可完成性保护。

19. EXPLAIN 与 Query Profile 分别回答什么问题？

EXPLAIN展示不执行SQL时的计划与估算；Profile记录实际执行的行数、耗时、内存、等待和Task差异，二者对照才能发现估算偏差与运行时瓶颈。

问题	EXPLAIN	Profile
扫描哪些分区	计划中的partitions/tablets	实际读取行数与字节
Join怎样分布	Broadcast/Shuffle等选择	Build/Probe、网络与Hash内存
是否倾斜	通常看不出真实程度	max/avg/min直接比较
RF是否有效	是否规划生成与应用	是否到达、过滤多少

得分关键词

计划与实际；估算；RowsProduced；InputRows；MemoryPeak；Wait；max/avg/min

常见误区：只看Profile中最耗时的算子就直接调它。它可能在处理上游制造的异常大数据。

20. 怎样定位慢查询的第一处异常节点？

从Scan沿数据流向上，将EXPLAIN估算与Profile实际行数、字节、时间和内存逐节点比较；第一次明显偏离业务常识或估算的位置，通常比最终最慢算子更接近根因。

标准顺序

- 记录SQL、Query ID、Session变量、数据规模与总耗时。
- 检查Scan分区/Tablet裁剪、谓词与读取列。
- 检查Join顺序、Build Side、分布和Runtime Filter。
- 检查Local Aggregate缩减率、Exchange字节与TopN。
- 在MergedProfile比较估算/实际与max/avg/min，再下钻异常BE/Task。
- 提出一个根因假设，一次修改一个因素。
- 重新EXPLAIN与Profile，并同时验证结果、延迟和资源。

Sort30
Join20
Key
JoinSort

得分关键词

从Scan向上；估算/实际；第一处偏离；max/avg/min；单变量；回归验证

常见误区：一次同时改SQL、统计、Hint、并行度和内存。即使变快，也无法知道真正原因。

20题一句话答案

题号	一句话
1	SQL定结果语义，物理计划决定读取、移动和计算路线。
2	逻辑是关系运算，物理是算法分布，Fragment是模板，Instance是运行副本。
3	RBO清理确定浪费，CBO用统计比较候选成本。
4	低估Build会把Broadcast的网络和Hash表按Instance成倍放大。
5	NDV是不同值数，Cardinality是节点行数，上游误差会污染下游选择。
6	Exchange改变数据分布，因此发送端和接收端自然分属不同Fragment。
7	MPP用多机，Pipeline调线程，向量化批量用CPU。
8	阻塞Task让出Worker，背压阻止中间数据无限堆积。
9	Partition、Tablet、列和Page从粗到细逐层跳过数据。
10	函数削弱范围推导，SELECT *阻止列裁剪。
11	小侧Build Hash表，大侧按Block Probe可降低内存与等待。
12	四种策略的核心差别是两侧数据移动量与布局约束。
13	倾斜是工作不均，膨胀是Join输出基数异常增大。
14	Exchange前先局部聚合，网络只传可合并状态。
15	TopN只维护N个候选；大OFFSET仍需保留大量前置候选。
16	IN精确、Bloom排除肯定不存在、MinMax排除范围外。
17	高并行换单查询延迟，也增加状态、同步并挤压并发。
18	内存超限先查计划和基数，盲目加内存只会推迟问题。
19	EXPLAIN看计划与估算，Profile看真实运行。
20	从Scan向上找第一次规模或耗时异常，并做单变量验证。

参考资料

答案依据第四章教程，并以 Apache Doris 3.0 与 3.x 官方文档中的稳定机制校验。

- [1] Query Optimizer
- [2] Statistics
- [3] Pipeline Execution
- [4] Runtime Filter
- [5] TopN Optimization
- [6] Data Skew
- [7] Query Profile
- [8] Query Memory